Quantum chemistry in a browser tab: a WebAssembly/WebGPU electronic-structure stack and a browser-tab swarm that scales Hartree–Fock to C_{60}

Ahmet Barış Günaydın
Independent researcher · github.com/abgnydn/webgpu-q

Abstract

We present **webgpu-q**, an electronic-structure stack — Hartree–Fock (RHF/UHF), MP2, CCSD, CCSD(T), density-functional theory across the LDA/GGA/hybrid ladder, and EE/IP/EA-EOM-CCSD — that runs entirely inside a web browser with no installation, no server, and no CUDA. We are not aware of a prior in-browser implementation at this method coverage. Numerical hot paths (electron-repulsion integrals, the auxiliary-basis density-fitting B-tensor, the Fock build) are hand-written in Rust compiled to WebAssembly with SIMD128; the methods themselves are ported from PySCF [1] with attribution, so the contributions are the delivery mechanism, the SIMD kernels, and the distribution layer. We introduce a **browser-tab swarm**: the density-fitted Fock build is partitioned by auxiliary index across N same-origin browser tabs that coordinate over `BroadcastChannel` and `SharedArrayBuffer`, and the partial Coulomb and exchange matrices are summed to reproduce the single-tab build to a relative error of order 10^{-15}. The swarm raises the single-tab memory ceiling that otherwise caps browser quantum chemistry, and scales Hartree–Fock to C_{60} **buckminsterfullerene (300 basis functions, STO-3G)** on a 16 GB consumer laptop, distributing the 1.82 GB three-index tensor across four tabs at 454 MB each. Energies are validated against PySCF — bit-for-bit where the algorithm is identical, and to $\leq 50\,\mu$Ha (HF) / ≤ 0.25 mHa (CCSD(T) vs FCI) otherwise. Speed is workload-dependent and is not the contribution: against PySCF (the only suite we benchmark), the WebAssembly+SIMD path is comparable-to-faster on minimal-basis HF/MP2/CCSD for small molecules but up to ∼two orders of magnitude slower on large correlated calculations, where native vectorized BLAS dominates. The contribution is delivery — full quantum chemistry reachable from a URL.

1 Introduction

Electronic-structure software is, almost without exception, native code: PySCF, Psi4, ORCA, Gaussian, and Q-Chem all ship as compiled binaries that require installation, a tuned BLAS, and — increasingly — a CUDA-capable GPU [2, 3, 4]. This install-and-compile barrier is a real obstacle to teaching, reproducibility, and casual exploration: a student cannot "try a CCSD(T) calculation" the way they can open a JSFiddle.

The browser has, separately, become a credible compute platform. WebAssembly delivers near-native throughput for numerical code; SharedArrayBuffer plus cross-origin isolation enables true multithreading; WebGPU exposes compute shaders; and WebRTC/BroadcastChannel enable peer-to-peer coordination. Browser volunteer-computing frameworks (Pando [5], Genet [6], QMachine [7]) have run generic map/stream workloads across browser tabs and devices. Web-based chemistry tools such as WebMO [8] and the Molecule Calculator [9] put a browser front-end on quantum chemistry, but the calculation runs server-side on a native backend. No published system runs

electronic structure client-side in the browser itself, and none distributes such a calculation across browser tabs (§5).

This paper closes that gap. Our contributions are:

1. **A complete browser-native electronic-structure stack** spanning HF through CCSD(T), DFT, and EOM-CCSD, validated bit-for-bit against PySCF.
2. **A WebAssembly+SIMD integral/Fock kernel** that brings benzene cc-pVDZ HF from 841 s (TypeScript baseline) to seconds.
3. **A browser-tab swarm** that partitions the density-fitted Fock build by auxiliary index across N tabs, validated bit-exact against the single-tab result, and that scales HF to C_{60}.
4. **A research-grade reproducibility harness** — every result records git SHA, hardware, WebGPU limits, seeds, and pass bars; the entire artifact is a URL a referee can re-run.

We are explicit about what is *not* novel: the chemistry methods are textbook and are ported from PySCF with Apache-2.0 attribution. The novelty is the delivery mechanism, the SIMD kernels, and the swarm distribution algorithm.

2 Methods

2.1 Hand-written vs. ported layers

Following an explicit engineering policy ("port, don't re-derive"), only the genuinely novel layer is hand-written: the WGSL compute shaders, the WebGPU dispatch and synchronization, the MPS browser memory bookkeeping, the WebAssembly integral kernels, and the swarm harness. The quantum-chemistry methods (HF, MP2, CCSD, CCSD(T), DFT functionals, EOM-CCSD σ-equations) are ported from PySCF with per-file attribution and a root `NOTICE` consolidating the Apache-2.0 provenance. Ported modules are validated by full-tensor element-wise diffs against brute-force reference implementations, not block-max metrics.

2.2 WebAssembly integral and Fock kernels

The electron-repulsion integrals (McMurchie–Davidson recursion), the three-index $(\mu\nu|P)$ and two-index $(P|Q)$ density-fitting integrals, and the Fock J/K contraction are implemented in Rust compiled to `wasm32-unknown-unknown` with `simd128` (f64x2). Per-thread scratch buffers are reused across SCF iterations to avoid GC churn; the K-build exploits the $K[\mu,\nu] = K[\nu,\mu]$ symmetry; the inner contraction loops use 2-lane f64 (f64x2) SIMD. Workers are pre-warmed before the SCF loop so iteration 1 does not pay WASM JIT + heap-growth cost on the critical path.

2.3 Auxiliary-basis density fitting

The rank-4 ERI tensor $(\mu\nu|\lambda\sigma)$ is replaced by a rank-3 B-tensor via $(\mu\nu|\lambda\sigma) \approx \sum_P B[\mu\nu, P]\, B[\lambda\sigma, P]$, with $B = V \cdot M^{-1/2}$ formed by a pivoted incomplete Cholesky factorization of the auxiliary metric M. This never materializes the n^4 tensor (10.4 GB on naphthalene cc-pVDZ) — the B-tensor is 303 MB. An auto-auxiliary generator (decontract + angular extension) removes the dependency on external aux-basis tables.

2.4 The browser-tab swarm

The density-fitted Fock build is linear in the auxiliary index P:

$$\begin{aligned}\gamma[P] &= \textstyle\sum_{\mu\nu} B[\mu\nu, P] \cdot D[\mu\nu] \\ J[\mu\nu] &= \textstyle\sum_{P} B[\mu\nu, P] \cdot \gamma[P] \\ X[P, \mu, \sigma] &= \textstyle\sum_{\lambda} B[\mu\lambda, P] \cdot D[\lambda, \sigma] \\ K[\mu, \sigma] &= \textstyle\sum_{P} \sum_{\sigma'} X[P, \mu, \sigma'] \cdot B[\sigma\sigma', P]\end{aligned}$$

Partitioning P into disjoint ranges and summing partial (J, K) therefore reproduces the full result. We verified this empirically: at 2/3/4/8 partitions the partial-sum Fock matrices match the single-slab build to relative error $\lesssim 10^{-15}$ (f64 accumulator-reorder noise).

Setup (once). The master computes the full B-tensor, partitions the auxiliary index range $[0, n_{\text{aux}})$ into N disjoint slices, and ships slice T to worker tab T. Each worker acknowledges receipt; the master then holds only its own slice. (A per-tab *independent* build — each tab forming only its aux-slice from scratch, so the full B never resides on any single tab — is designed but not yet implemented; §4.)

Per SCF iteration. The master drives an otherwise-unchanged Roothaan–Hall/DIIS loop, with the Fock build replaced by a `customJKBuilder` callback:

```
master: worker T (xN):
  broadcast D ------------------------> receive D
  compute own partial (J_0, K_0) compute partial (J_T, K_T)
    on slice 0 on slice T (x2 inner
                                    SAB workers)
  await N-1 partials <---------------- post (J_T, K_T)
  J = sum_T J_T ; K = sum_T K_T
  G = J - (1/2) K -> resume SCF
```

Because each (J_T, K_T) is the contribution of a disjoint P-range and the Fock build is linear in P (§2.4 equations), the summed (J, K) equals the single-slab result up to f64 accumulator-reorder noise. The master overlaps its own slice computation with the workers' (it does not block on the broadcast), so the per-iteration wall-time is set by the slowest single tab plus one round-trip, not the sum.

Transport. Control messages (the D broadcast, the partial-result gather, slice-distribution acknowledgements) travel over `BroadcastChannel`. For same-machine multi-tab runs the large B-slice payloads and per-iteration halo-free state are shared zero-copy via `SharedArrayBuffer`; `BroadcastChannel`'s structured-clone is reserved for the small D ($n \times n$ f64) and the (J, K) partials. A WebRTC transport (via a PeerJS broker with STUN) carries the same protocol across machines, at the cost of structured-clone serialization on every message.

Topology. The per-tab JK build itself parallelizes across inner SAB workers. On a 10-core M2 Pro, a 4-tab × 2-inner-worker layout (8 compute threads spread over 4 V8 processes) outperformed both 1-tab × 8-worker and 2-tab × 4-worker at equal thread count — independent processes incur less scheduling contention than one process driving eight workers against a shared WASM heap.

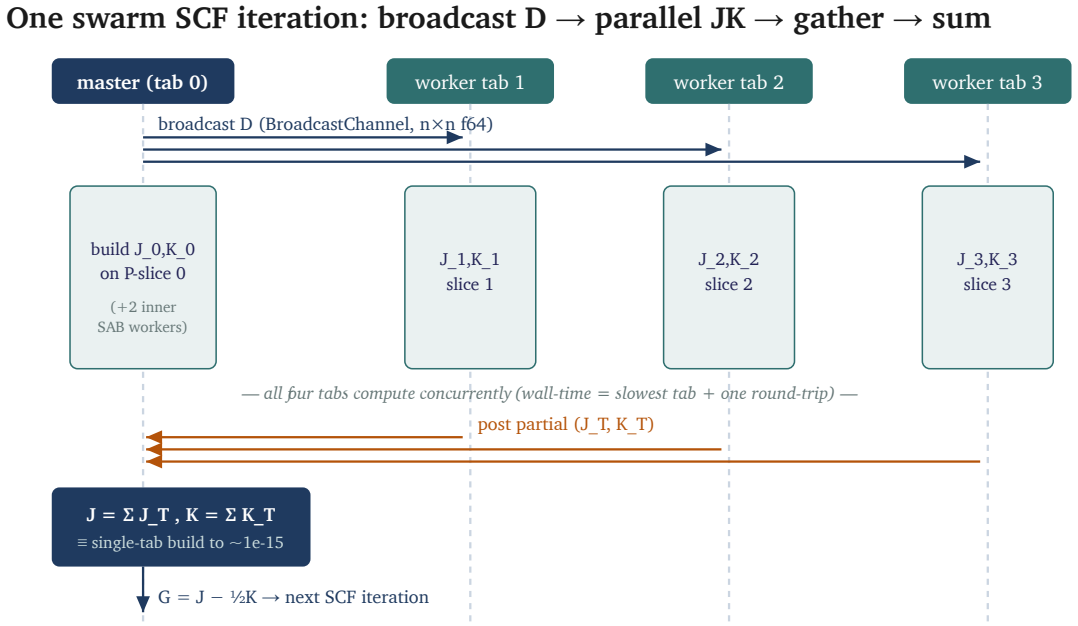

Figure 1: One swarm SCF iteration. The master broadcasts the density matrix D to all worker tabs over BroadcastChannel; each tab (including the master) builds its partial Coulomb/exchange contribution on its own disjoint auxiliary-index slice of B, concurrently; workers post their partials back; the master sums them — reproducing the single-tab Fock build to ~1e-15 — and resumes the SCF loop. Per-iteration wall-time is the slowest tab plus one round-trip, not the sum.

3 Results

3.1 Validation

Where methods overlap, energies match PySCF bit-for-bit. HF agrees to $\leq 50\,\mu$Ha with spherical-d; CCSD(T) reproduces FCI to ≤ 0.25 mHa on STO-3G systems; GPU↔CPU agreement is $\approx 10^{-10}$ Ha (f32 reduction noise, six orders below chemical accuracy of 1.594 mHa). End-to-end, the distributed-swarm energy reproduces the single-tab reference to 4.5×10^{-13} Ha on benzene (the committed swarm artifact records both energies), far inside the 10^{-7} convergence tolerance the spec asserts; the partitioned Fock build itself matches the single-slab build below the 10^{-12} pass bar (next paragraph).

Correctness is enforced by reference-grounded gates that run in continuous integration on every push (alongside the end-to-end browser benchmarks driven through headless Chromium), not by aggregate test counts. The EOM-CCSD family is gated by *full-tensor* brute-force diagnostics — the complete σ-matrix is differenced element-by-element against an exact reference (e.g. a 14×14 LiH $M_{\text{mine}} - M_{\text{exact}}$ diff), not by block-max metrics that can hide structural bugs. Methods ported from PySCF were accepted only after this element-wise diff collapsed to numerical noise. DMRG/MPS results are cross-checked against ITensor at $N = 8$ to f64 precision [10], and 1D-model limits against the analytic Pfeuty (TFIM) and Bethe (Heisenberg) results. The swarm's auxiliary-index partitioning was verified to reproduce the single-slab Fock build to below the committed 10^{-12} relative-error pass bar — observed $\lesssim 10^{-15}$ — at 2-, 3-, 4-, and 8-way splits before any multi-tab run.

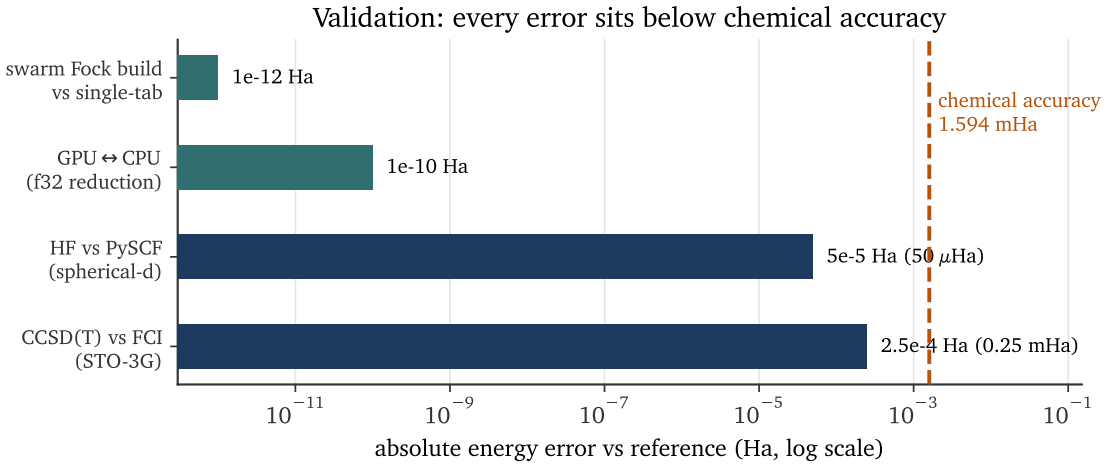

Figure 2: Accuracy ladder. Each bar is the absolute energy error against its reference on a log axis; the dashed line is the Pople chemical-accuracy threshold (1 kcal/mol = 1.594 mHa). The two algorithmic-precision checks (swarm Fock build vs single-tab, ∼1e-12 Ha; GPU↔CPU f32 reduction, ∼1e-10 Ha) sit six to nine orders below it, and the method errors (HF vs PySCF $\leq 50\,\mu$Ha, CCSD(T) vs FCI ≤ 0.25 mHa) remain sub-chemical.

3.2 Single-tab performance (M2 Pro, Chromium, COI enabled)

Naphthalene cc-pVDZ ($n = 190$) HF SCF, cumulative single-tab optimization:

optimization	SCF wall
parallel V-build + B back-substitution	baseline
reuse JK worker scratch buffers	43 s → 20 s
exploit K symmetry	20 s → 17 s

These two kept optimizations cut the naphthalene SCF wall from 43 s to 17 s (≈ 2.5×). On the same molecule the 4-tab swarm then runs the distributed SCF in 12.0 s versus 22.6 s for the single-tab parallel build (1.9×; both recorded in the committed naphthalene artifact, §3.3). (A 4× unroll of the SIMD inner loop was also attempted and reverted — 17.88 vs 18.01 s on benzene, within ±1% noise — since the 2-lane f64 SIMD already saturates the pipeline at these problem sizes.)

3.3 Swarm scaling across the acene series and to C_{60}

All energies converged and are deterministic run-to-run, on a 16 GB M2 Pro:

molecule	basis	n	n_{aux}	path	SCF wall†	E (Ha)
benzene	cc-pVDZ	120	662	2-tab	5.6 s	−230.72269
naphthalene	cc-pVDZ	190	1048	4-tab × 2-inner	12 s	−383.38458
anthracene	STO-3G	80	708	4-tab	2.1 s	−526.92320
pentacene	STO-3G	124	1091	4-tab × 2-inner	40 s	−821.63271
C_{60}	STO-3G	300	2520	4-tab × 2-inner	539 s	**−2244.10176**

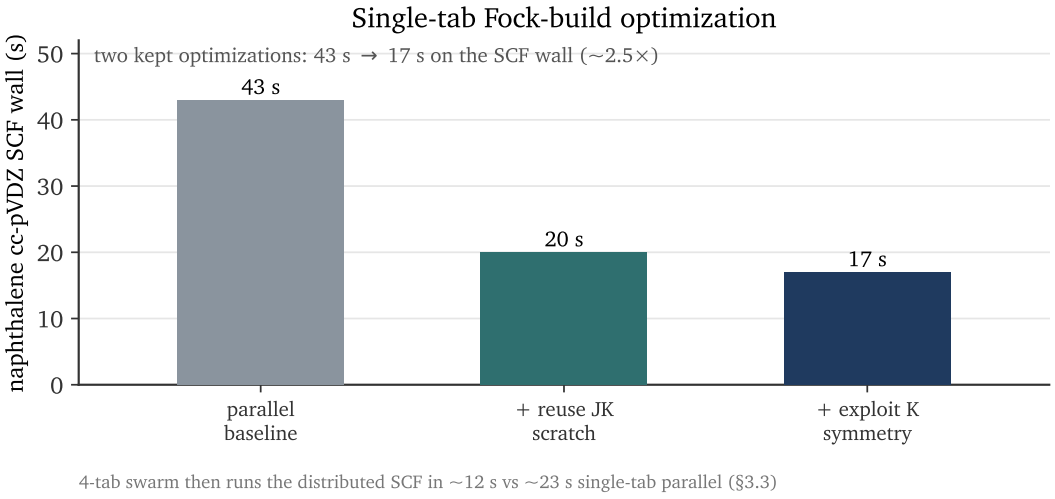

Figure 3: Single-tab Fock-build optimization on naphthalene cc-pVDZ. Reusing per-thread JK scratch buffers across SCF iterations and exploiting the $K[\mu,\nu] = K[\nu,\mu]$ symmetry cut the SCF wall from 43 s to 17 s ($\approx 2.5\times$). The 4-tab swarm then runs the distributed SCF in ≈ 12 s versus ≈ 23 s single-tab parallel (§3.3).

[†]Wall is the distributed SCF loop (the per-iteration broadcast-D / parallel-(J,K) / gather), the swarm's actual work; the one-time integral and auxiliary-DF build is excluded (it is machine-load sensitive). Energies, n, n_{aux}, and iteration counts are deterministic and each row is backed by an environment-stamped JSON artifact under `experiments/results/<date>/swarm/` with the full per-phase breakdown. All five rows are committed (M2 Pro local runs); naphthalene also reproduces on the CI runner. C_{60} runs only locally — its ∼1.8 GB B-tensor build peaks above the CI runner's ∼2 GB WASM-heap cap, so CI reproduction awaits the streaming-kernel rewrite noted in §4.

The naphthalene swarm (12 s) is faster than the single-tab parallel path (22.6 s) because four independent V8 processes incur less coordination overhead than one process scheduling eight SAB workers.

The largest case, C_{60} buckminsterfullerene, is the clearest demonstration of the swarm's purpose: its full HF SCF converges in 9 iterations inside four browser tabs, with the 1.82 GB three-index tensor distributed at 454 MB per tab — well under any single tab's ∼2 GB SharedArrayBuffer ceiling, which a single-tab build cannot satisfy (§3.4).

3.4 The memory wall and where it moves

A single tab caps at ≈ naphthalene ($n \approx 220$) cc-pVDZ; the 4-tab swarm reaches C_{60} ($n = 300$). The master-builds-then-partitions design peaks at ∼3 GB during the V+B build for C_{60}; a per-tab independent B-build (each tab builds only its aux-slice from scratch) would lower the per-tab peak and is designed but not yet implemented.

4 Honest limitations

Per our research-grade discipline, failures are reported, not hidden:

- **Multireference wall.** Linear acenes beyond pentacene develop open-shell-singlet/polyradical

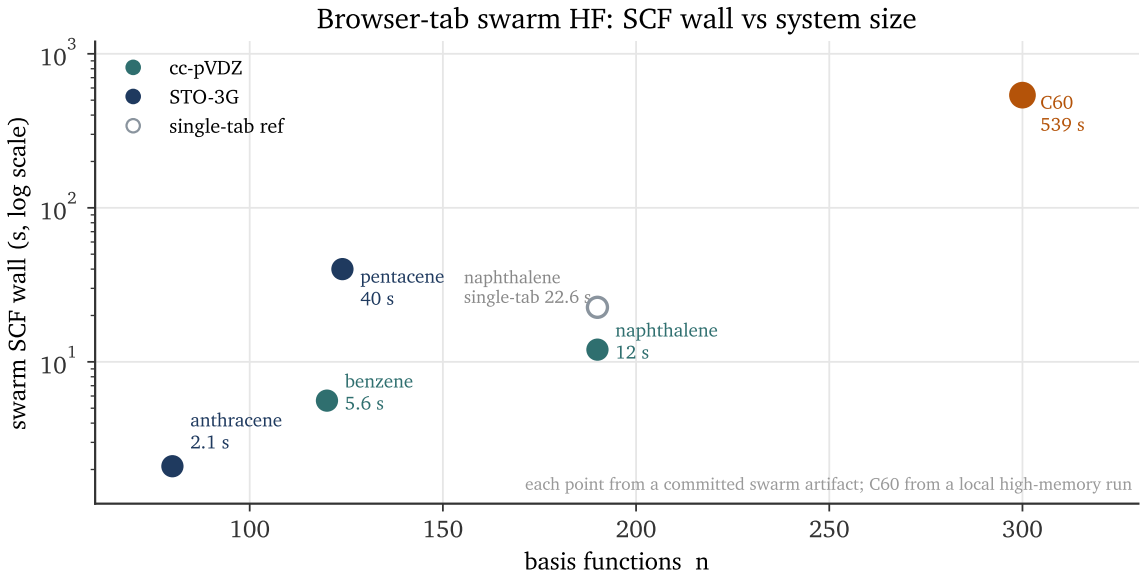

Figure 4: HF SCF wall-time vs basis-function count across the acene series and C_{60}, colored by basis set, log-time axis; each point is from a committed swarm artifact. The swarm reaches C_{60} (300 basis functions); the open marker shows naphthalene's single-tab parallel SCF (22.6 s) above its 4-tab swarm SCF (12 s), a 1.9× intra-machine gain. The single-tab SharedArrayBuffer ceiling is basis-dependent ($\sim n = 220$ for cc-pVDZ, higher for STO-3G), so it is stated in text rather than drawn as one line.

character; single-determinant RHF does not reliably converge for hexacene/heptacene (and convergence at octacene is luck, not signal). The swarm runs the full SCF to completion and returns finite energies, but RHF convergence there is method-limited — UHF / spin-projected references are the fix.

- **Anthracene cc-pVDZ basin selection.** A delayed-DIIS recipe (damping 0.2, DIIS start at iter 8) avoids divergence but converges to a spurious lower-energy state; MOM / SAD-initial-guess is required for the physical ground state.
- **Density fitting gives no speedup yet** — current CD-DF still builds the 4-index tensor before decomposing; aux-basis 3-index DF is the real win.
- C_{60} **runs only at STO-3G**; cc-pVDZ at $n \approx 600$ exceeds even the swarm-distributed master's build memory.

5 Related work

GPU-accelerated native quantum chemistry is mature: GPU4PySCF [2], QUICK [3], and Velox-Chem [4] implement HF/DFT/Fock on CUDA. None runs in a browser. Browser volunteer computing — Pando [5], Genet [6], QMachine [7] — distributes generic workloads across tabs and devices but has not run electronic structure. WebGPU is an actively characterized compute target [11]. Web-based quantum-chemistry tools do exist — WebMO [8] and the Molecule Calculator [9] — but they are server-client front-ends: the browser is a molecular editor and job monitor, while the SCF runs on a

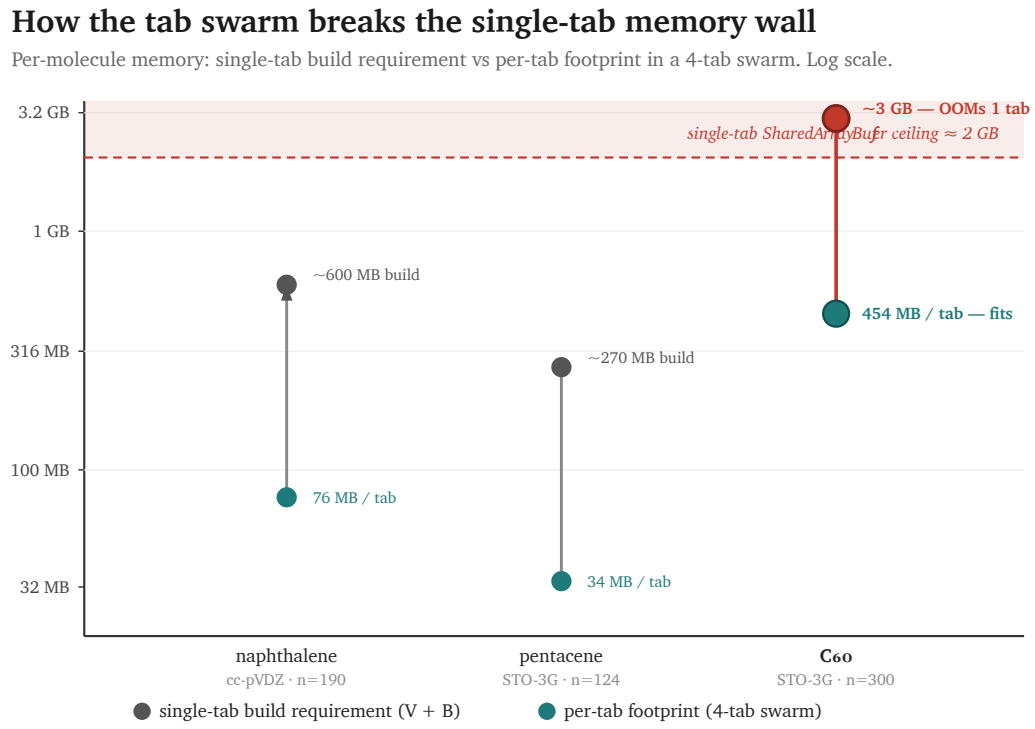

Figure 5: Per-molecule memory on a log scale: the single-tab build requirement (V+B, grey) vs the per-tab footprint in a 4-tab swarm (teal), with the ∼2 GB single-tab SharedArrayBuffer ceiling as a shaded band. naphthalene and pentacene fit a single tab; C_{60}'s ∼3 GB build requirement exceeds the ceiling — it cannot run in one tab — but its 454 MB per-tab slice in the 4-tab swarm sits well under, which is what makes browser-tab C_{60} Hartree–Fock possible.

native backend (GAMESS, Gaussian, etc.), and the client-side methods are limited to RHF/STO-3G property estimation. webgpu-q instead runs the entire HF/MP2/CCSD(T)/DFT/EOM stack *in* the browser, client-side, with no backend. On a literature search current to June 2026 we find no published in-browser (client-side) electronic-structure implementation at this method coverage, and none that distributes such a calculation across browser tabs. Best-functional and benchmark context follows the GMTKN55 database [12]; experimental reference data are from the NIST CCCBDB [13].

6 Software availability and reproducibility

Source. The complete source is at `github.com/abgnydn/webgpu-q`, public, MIT-licensed for original code; portions ported from PySCF are Apache-2.0 with per-file attribution consolidated in a root `NOTICE` (Apache-2.0 §4(d)). The WebAssembly integral kernels are Rust in `wasm-eri/`; the swarm in `src/parallel/`; the chemistry in `src/chemistry/`.

Running it. No installation, no account, no GPU driver: the simulator runs in any WebGPU-capable Chromium with cross-origin isolation (COOP/COEP) enabled. The swarm requires N same-origin tabs; for the cross-machine variant a PeerJS broker + STUN suffices. A reader can reproduce a single-molecule HF run from a browser tab; the multi-tab swarm runs are reproduced

by opening N tabs of the same origin.

Reproducibility harness. Every experiment emits a JSON artifact recording the git commit SHA, `navigator.userAgent`, `adapter.info` (vendor/architecture/device), WebGPU device limits, OS, a UTC ISO-8601 timestamp, and the echoed `protocol`, `hypothesis`, `seed`, `warmup`, `trials`, and `pass_bar`. No `Math.random()` appears on any experiment path — every random draw is a named deterministic seed. Wall-clock is measured with `performance.now()` bracketed by forced GPU sync (a mapped readback) because `queue.submit` is non-blocking; 5 warmup samples are discarded and 20 retained, reporting median/p10/p90/p99/std/IQR. Failing configurations are committed with `status:"fail"` and a diagnosis rather than silently rerun — the honest-negative discipline that surfaced, for example, the acene multireference wall and the anthracene cc-pVDZ basin-selection issue reported in §4. The committed `experiments/results/<date>/` artifacts are the source of the level-1/2/3 and validation numbers; the swarm scaling table (§3.3) is produced by the end-to-end browser benchmark specs under `e2e/`, each of which now emits the same environment-stamped JSON artifact per run (git SHA, `adapter.info`, energy, n, n_{aux}, iterations, and per-phase wall-time) under `experiments/results/<date>/swarm/`.

Generative-AI disclosure. Portions of the software, its documentation, and this manuscript were drafted with the assistance of a large language model used as a coding and writing aid. All AI-assisted output was reviewed by the author; correctness of code was enforced by the test suite, brute-force diagnostics, and PySCF cross-validation described in §3.1, and every quantitative claim in this paper is traceable to a committed experiment artifact rather than to model output.

Statements. Sole author; no competing interests; no external funding. Archived at Zenodo, concept DOI 10.5281/zenodo.20494382 (resolves to the latest version).

7 Conclusion

A full electronic-structure stack runs in a browser tab, validated against PySCF, and a browser-tab swarm scales it to C_{60} without native code, a data-center GPU, or an install. It does not aim to beat native suites on raw speed — on large correlated calculations it is far slower — and its value lies elsewhere: in removing the install, hardware, and cost barriers between a learner and a real calculation, and in showing that the browser's memory ceiling is not a hard limit but one that a tab swarm can raise. The entire artifact is a URL, and a referee can re-run every number reported here.

References

[1] Qiming Sun et al. PySCF: the Python-based simulations of chemistry framework. *WIREs Computational Molecular Science*, 8:e1340, 2018. DOI: 10.1002/wcms.1340.

[2] Rui Wu, Qiming Sun, et al. Enhancing GPU acceleration in the Python-based simulations of chemistry framework. *Journal of Physical Chemistry A*, 2024. DOI: 10.1021/acs.jpca.4c05876. Preprint arXiv:2407.09700.

[3] Madushanka Manathunga, Yipu Miao, Dawei Mu, Andreas W. Götz, and Kenneth M. Merz. Parallel implementation of density functional theory methods in the quantum interaction computational kernel (QUICK) program. *Journal of Chemical Theory and Computation*, 16:4315–4326, 2020. DOI: 10.1021/acs.jctc.0c00290.

[4] Xin Li, Mathieu Linares, and Patrick Norman. VeloxChem: GPU-accelerated Fock matrix construction enabling complex polarization propagator simulations of circular dichroism spectra of G-quadruplexes. *Journal of Physical Chemistry A*, 129(2):633–642, 2025. DOI: 10.1021/acs.jpca.4c07510.

[5] Erick Lavoie and Laurie Hendren. Pando: Personal volunteer computing in browsers. *arXiv preprint*, arXiv:1803.08426, 2018.

[6] Erick Lavoie, Laurie Hendren, et al. Genet: A quickly scalable fat-tree overlay for personal volunteer computing using WebRTC. *arXiv preprint*, arXiv:1904.11402, 2019.

[7] Sean R. Wilkinson and Jonas S. Almeida. QMachine: Commodity supercomputing in web browsers. *BMC Bioinformatics*, 15:176, 2014. DOI: 10.1186/1471-2105-15-176.

[8] William F. Polik and J. R. Schmidt. WebMO: Web-based computational chemistry calculations in education and research. *WIREs Computational Molecular Science*, 12(1):e1554, 2022. DOI: 10.1002/wcms.1554.

[9] Jan H. Jensen and Jimmy C. Kromann. The molecule calculator: A web application for fast quantum mechanics-based estimation of molecular properties. *Journal of Chemical Education*, 90(8):1093–1095, 2013. DOI: 10.1021/ed400164n.

[10] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011. DOI: 10.1016/j.aop.2010.09.012.

[11] Anonymous. Characterizing WebGPU dispatch overhead for LLM inference across four GPU vendors, three backends, and three browsers. *arXiv preprint*, arXiv:2604.02344, 2026.

[12] Lars Goerigk, Andreas Hansen, Christoph Bauer, Stephan Ehrlich, Asim Najibi, and Stefan Grimme. A look at the density functional theory zoo with the advanced GMTKN55 database for general main group thermochemistry, kinetics and noncovalent interactions. *Physical Chemistry Chemical Physics*, 19:32184–32215, 2017. DOI: 10.1039/C7CP04913G.

[13] NIST. NIST computational chemistry comparison and benchmark database, NIST standard reference database number 101, release 22. `https://cccbdb.nist.gov/`, 2022.